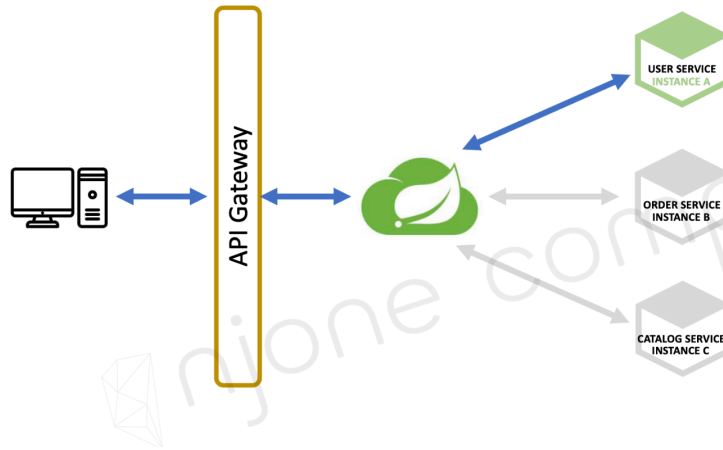


WEEK_5

Users MicroService(1)



프로젝트 생성 (user-service)

Dependencies

- Spring Boot DevTools
- Lombok
- Spring Web
- Eureka Discovery Client
- Spring Data JPA
- H2 Database

<https://github.com/bum0w0/spring-cloud-msa-study/commit/4760e6c41c0938006e9271ca01dda5b520b2e789>

사용자 추가 (회원가입)

- POST /users
- UserController ↔ UserService ↔ UserRepository 흐름
- RequestUser(RequestDTO), UserDTO, UserEntity 데이터 교환

사용자 추가 구현 1 (with JPA)

- UserServiceImpl class에 createUser() 메소드 추가

```
@Override
public UserDto createUser(UserDto userDto) {
    userDto.setUserId(UUID.randomUUID().toString());

    ModelMapper mapper = new ModelMapper();
```

```

mapper.getConfiguration().setMatchingStrategy(MatchingStrategies.STRICT);
UserEntity userEntity = mapper.map(userDto, UserEntity.class);
userEntity.setEncryptedPwd("encrypted_password");

userRepository.save(userEntity);

UserDto returnUserDto = mapper.map(userEntity, UserDto.class);

return returnUserDto;
}

```

Http Status Code 변경

```

@PostMapping("/users")
public ResponseEntity<ResponseUser> createUser(@RequestBody RequestUser user) {
    ModelMapper mapper = new ModelMapper();
    mapper.getConfiguration().setMatchingStrategy(MatchingStrategies.STRICT);

    UserDto userDto = mapper.map(user, UserDto.class);
    userService.createUser(userDto);

    ResponseUser responseUser = mapper.map(userDto, ResponseUser.class);

    return ResponseEntity.status(HttpStatus.CREATED).body(responseUser);
}

```

Spring Security 연동

- Step 1: 애플리케이션에 **spring security jar**을 Dependency에 추가
- Step 2: **WebSecurityConfigurerAdapter**를 상속받는 **Security Configuration** 클래스 생성
- Step 3: Security Configuration 클래스에 **@EnableWebSecurity** 추가
- Step 4: Authentication → **configure(AuthenticationManagerBuilder auth)** 메서드를 재정의
- Step 5: Password encode를 위한 **BCryptPasswordEncoder** 빈 정의
- Step 6: Authorization → **configure(HttpSecurity http)** 메서드를 재정의

스프링 시큐리티 6.x 버전으로 넘어오면서 몇몇 기능은 권장되지 않음. 따라서, Step 2, 4, 6 은 실습에서 제외

- Step 7: **SecurityFilterChain**을 반환하는 **configure** 빈 추가

BCryptPasswordEncoder

- password 해싱을 위해 Bcrypt 알고리즘 사용
- 랜덤 Salt 부여 및 여러 번 Hash를 적용한 암호화 방식

UserServiceImpl.java

```
@Autowired
BCryptPasswordEncoder passwordEncoder;

@Override
public UserDto createUser(UserDto userDto) {
    userDto.setUserId(UUID.randomUUID().toString());
    userDto.setEncryptedPwd(passwordEncoder.encode(userDto.getPwd()));
}
```

UserServiceApplication.java

```
@Bean
public BCryptPasswordEncoder bCryptPasswordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Utils.java (로그인 암호 생성)

```
public class Utils {
    public static void main(String[] args) {
        System.out.println(new BCryptPasswordEncoder().encode("password"));
    }
}
```

↓

```
$2a$10$4dUx4.6y8DnBpxhAFfj6lublA5FeyMqQQM67XOogUNIDTdQ6nw4Qy
```

↓

```
security:
  user:
    name: user
    password: $2a$10$4dUx4.6y8DnBpxhAFfj6lublA5FeyMqQQM67XOogUNIDTdQ6nw4Qy
```

Security 및 Bcrypt 적용 후 API 테스트

